GODOT: VERSIONAMENTO DE CÓDIGO

INTRODUÇÃO

Autor: Yuji

Este documento tem como objetivo apresentar, de forma clara e organizada, os conceitos fundamentais de versionamento de código utilizando **Git** e **GitHub**, com foco em projetos desenvolvidos na **Godot Engine**.

Ao longo deste material, serão abordados:

- A importância do versionamento de código
- O funcionamento do Git e do GitHub
- Como configurar versionamento em projetos Godot
- Fluxo básico de trabalho com commits e repositórios
- Boas práticas para trabalho em equipe
- Uso de branches e Pull Requests
- Gerenciamento de arquivos grandes com Git LFS

Este guia serve como base para padronizar o uso de versionamento dentro da equipe, promovendo organização, segurança e colaboração eficiente no desenvolvimento de projetos.

O documento é uma pesquisa de um dos integrantes do grupo de desenvolvimento de jogos, **Obscura**.

Referência

Link: https://www.youtube.com/watch?v=1vWzOnu4Nq0&list=WL&index=53&t=7s

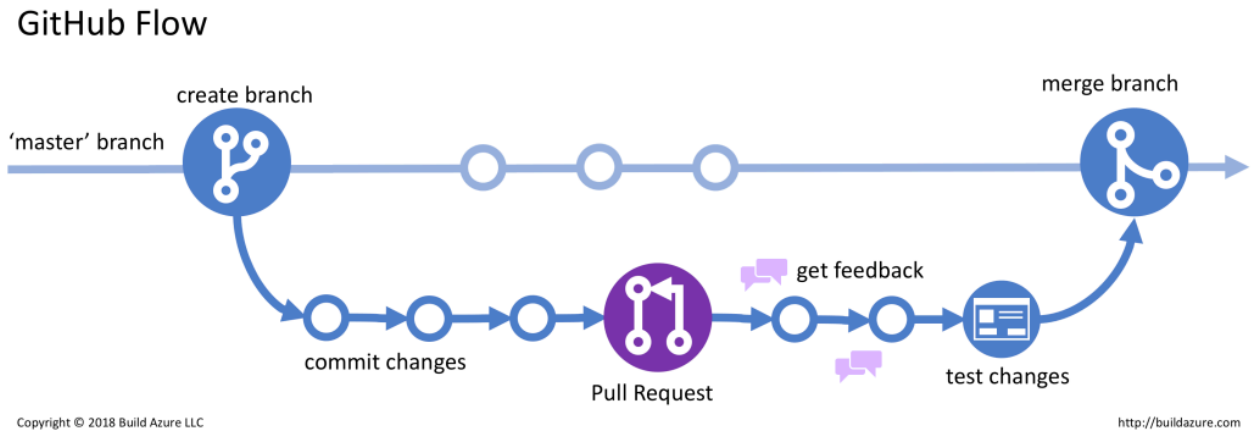
O que é versionamento de código?

O versionamento de código é crucial para desenvolvedores de código e equipes trabalharem de forma eficiente. Ele permite:

- Rastrear todas as alterações feitas no código
- Recuperar versões anteriores em caso de erro
- Trabalhar em equipes simultaneamente no mesmo projeto

Por que usar Git?

O **Git** é um **sistema de controle de versão distribuído** que permite rastrear todas as alterações feitas em seu código ao longo do desenvolvimento. Por exemplo, em caso de algum erro na versão atual, é possível retornar para as anteriores.



Um grande benefício do Git é a possibilidade de trabalhar no mesmo código simultaneamente em equipe.

Principais vantagens:

- Controle completo do histórico
- Possibilidade de desfazer alterações
- Trabalho simultâneo em equipe
- Versionamento local (sem depender da internet)

Por que usar Github?

Para compartilhar o nosso projeto Git, precisamos de uma plataforma de hospedagem como o **Github**. Ele é um serviço online que guarda os repositórios de códigos criados com o Git. Ele funciona como:

- Armazenamento em nuvem para projetos
- Ferramenta de colaboração em equipe
- Histórico visual de commits e alterações

Também é considerado uma "**rede social de devs**".

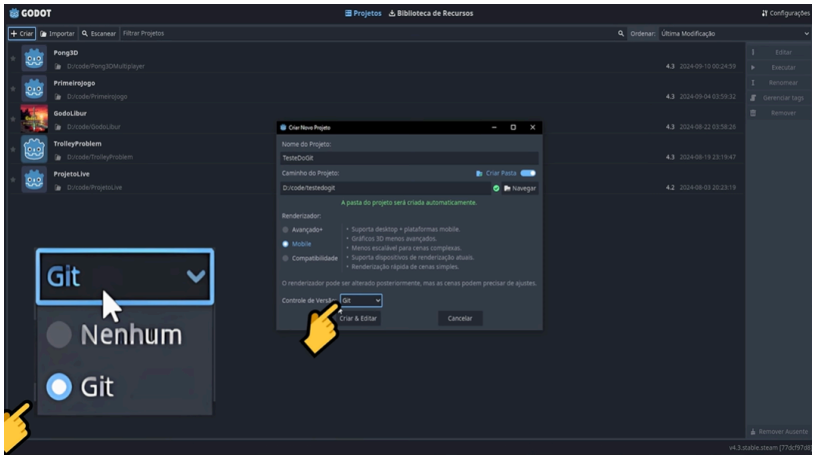
Como fazer o versionamento de código do Godot?

Primeiro vamos instalar o **Github Desktop**. É uma ferramenta que gerencia o Git de maneira simplificada, integrada diretamente com o Github. É uma alternativa para subir o código no repositório.

Como colocar o Git no projeto Godot?

Criando um novo projeto

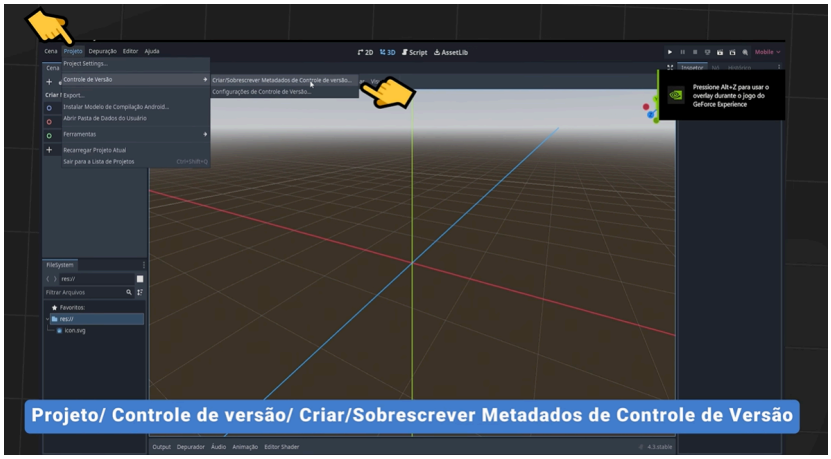
Ao criar um novo projeto, selecione a opção no **Controle de Versão:** Git.



Projeto existente

Para adicionar o Git como a ferramenta de controle de versão, basta clicar em:

Projeto → Controle de versão → Criar/Sobrescrever Metadados de Controle de Versão

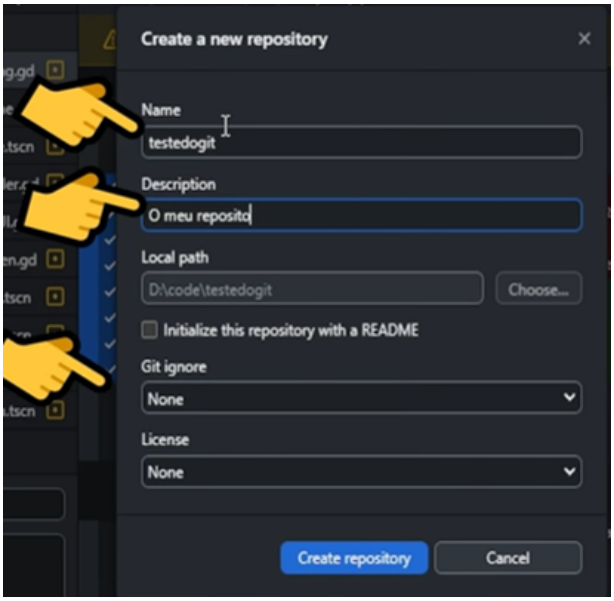


Criando repositório local

1. Abra o GitHub Desktop
2. Clique em: **File** → **Add Local Repository**
3. Selecione a pasta do projeto
4. Clique em **Create a Repository**

Ao criar:

- Defina nome e descrição
- Adicione um `.gitignore` (selecionar Godot)
- Escolha uma licença



A licença é importante porque todo código é considerado **propriedade intelectual** e precisamos de um documento que reconheça o autor e regule o seu uso para quem pode utilizar e para quais fins. Uma licença recomendada é a **MIT License**, por ser simples e permissiva.

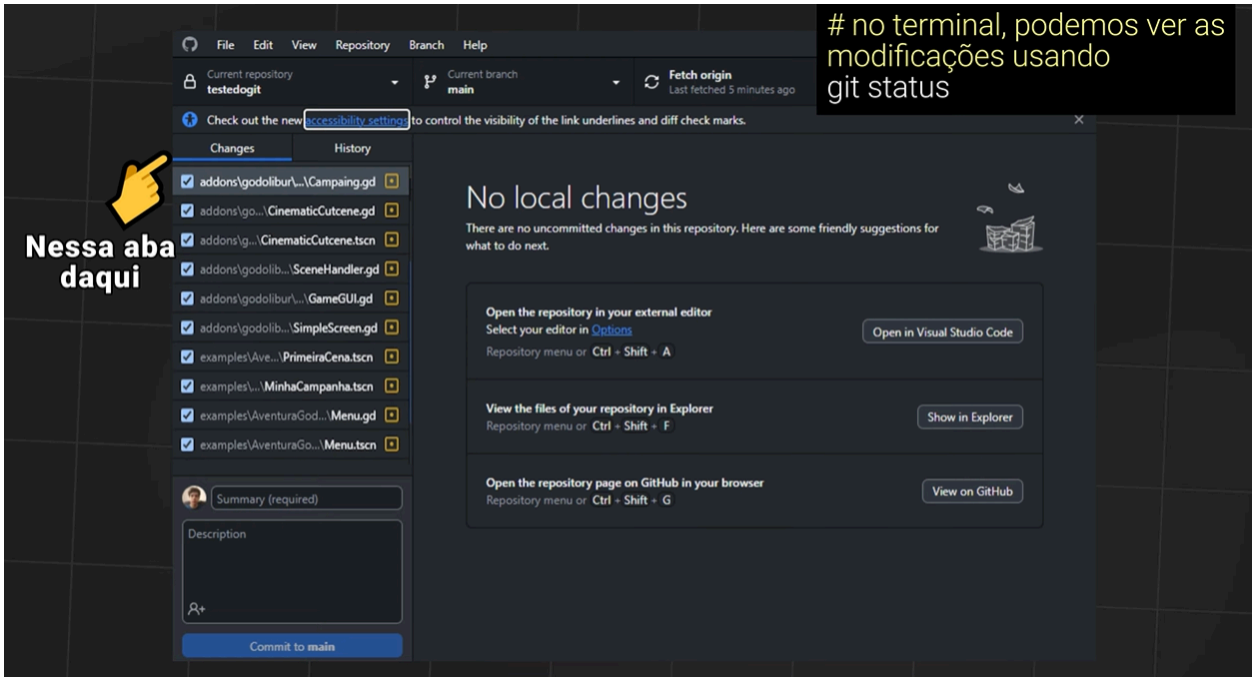
Criando repositório no Github

Agora com o repositório criado, é possível controlar os arquivos do seu código. Agora, vamos publicá-lo:

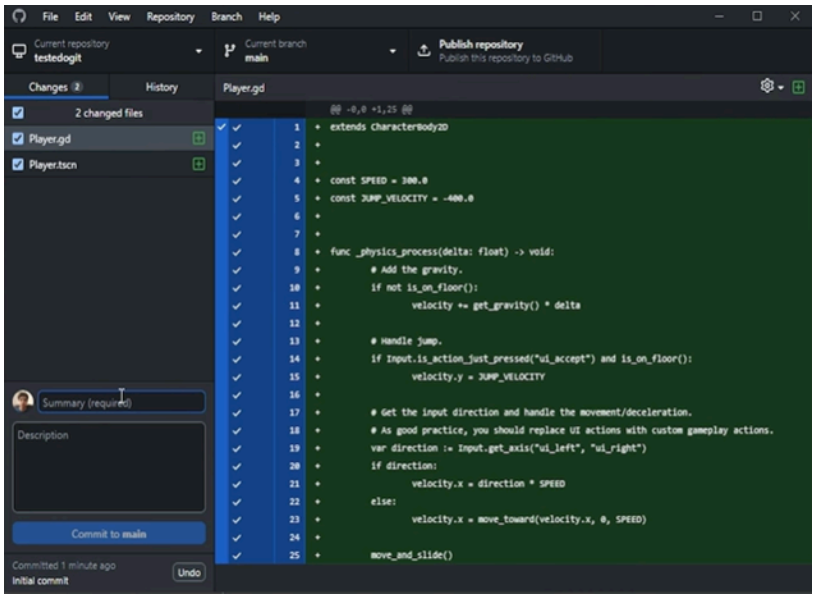
1. Vá até a aba **Changes**
2. Registre as alterações com um **commit**
3. Clique em **Publish Repository**

Configurações disponíveis:

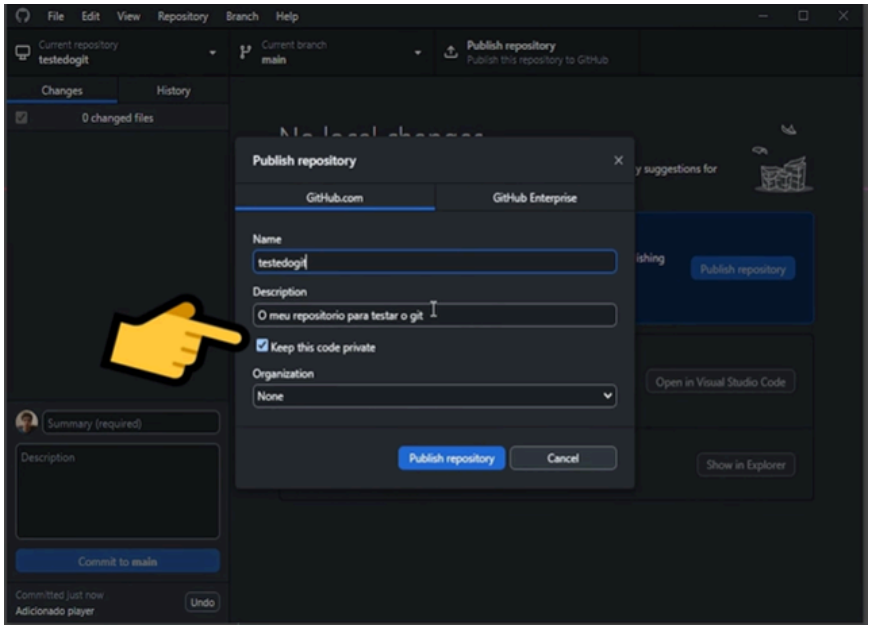
- Nome do repositório
- Descrição
- Público ou privado
- Organização (opcional)



Para registrar essas modificações, deve-se criar um **Commit**, uma atualização organizada de acordo com o objeto da mudança e arquivos selecionados.



Ao registrar o commit, ele aparecerá no **History** (Histórico). Após o registro, precisamos atualizar o repositório no Github com as mudanças locais, além de publicá-lo:



E assim seu projeto estará salvo no Github.

Arquivo .gitignore

O arquivo **.gitignore** define quais arquivos não devem ser enviados ao repositório, como a pasta **.godot** (arquivos temporários).

Fluxo de Versionamento Git

E assim é o fluxo principal do Git e Github:

- Trabalha num **repositório local** na sua máquina.
- Interage com o projeto.
- Quando tudo estiver pronto, “empacota” as suas mudanças com **commits** para enviar no repositório na nuvem.

Repositório

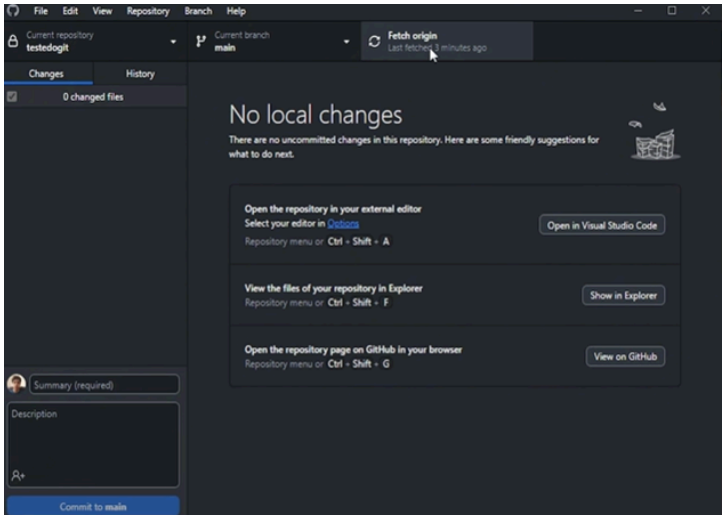
No **Github**, além de poder visualizar os arquivos, podemos ver todas as mudanças/commits feitas, quando e onde foram feitas.

Estrutura

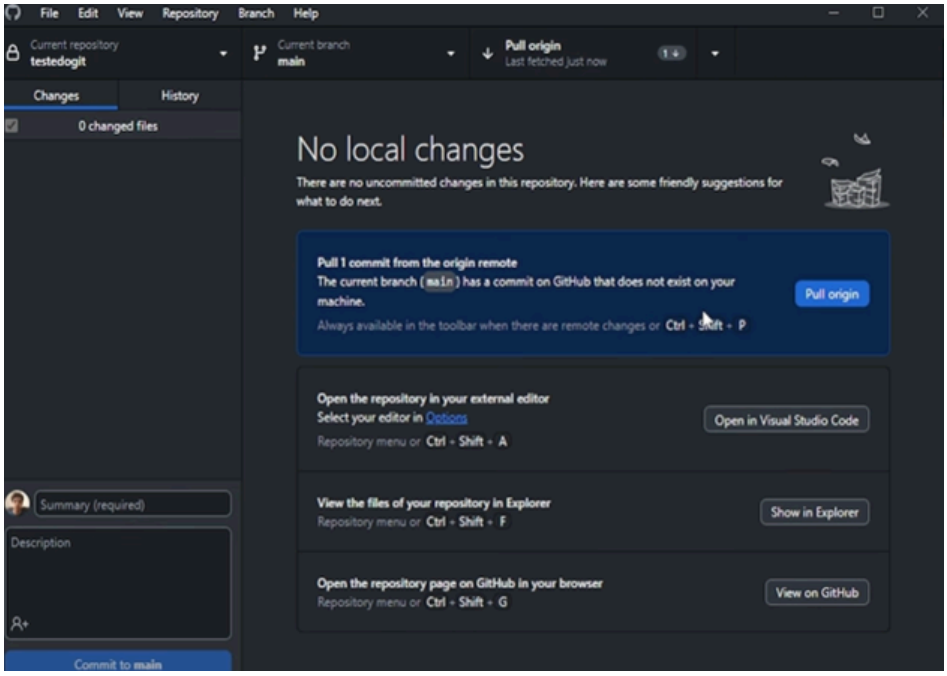
- **README.md:** Documento principal com descrição do projeto
- **History:** Histórico de commits
- **Changes:** Alterações atuais

Atualizações do repositório

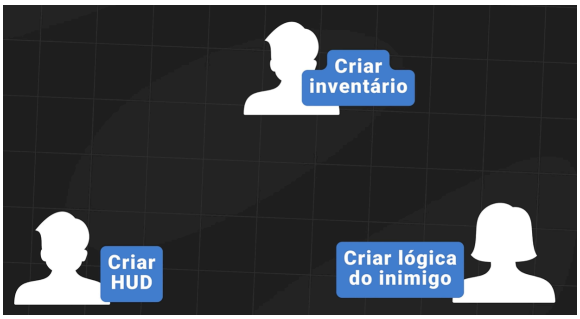
Quando houver qualquer modificação que não seja na sua máquina local, é preciso pedir permissão para alterações no **Github Repository**, utilizando o botão “**Fetch origin**”:



Se tiver alterações, aparecerá o **Pull origin** para puxá-las ao repositório local:



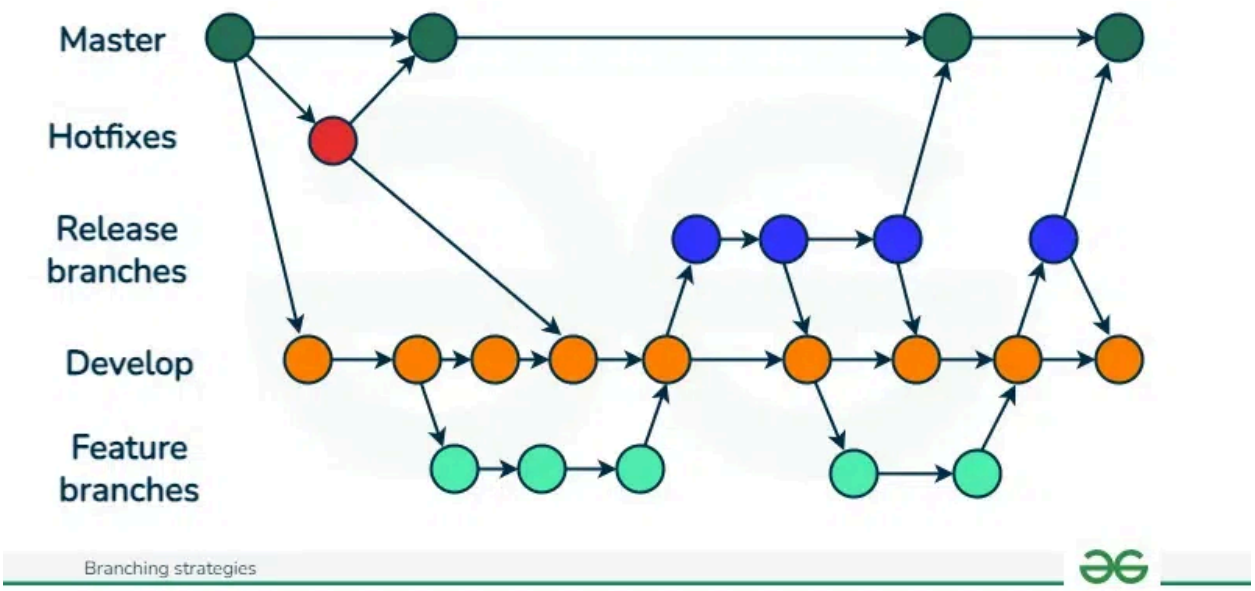
Integração no Trabalho em Equipe



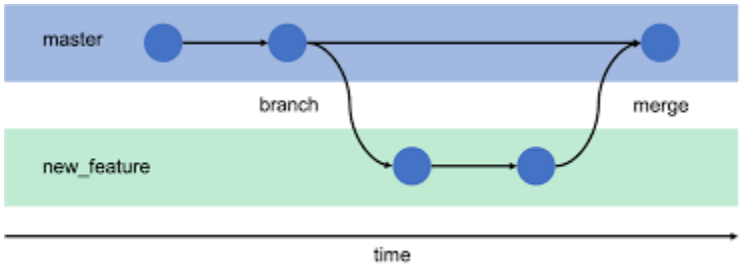
Vamos supor que você precise dividir as tarefas entre os integrantes de sua equipe e que cada um esteja encarregado de uma mecânica específica.

Branches

Primeiro, vamos entender o que são **branches**. Elas são bifurcações/ramificações de uma árvore com diferentes versões do código.



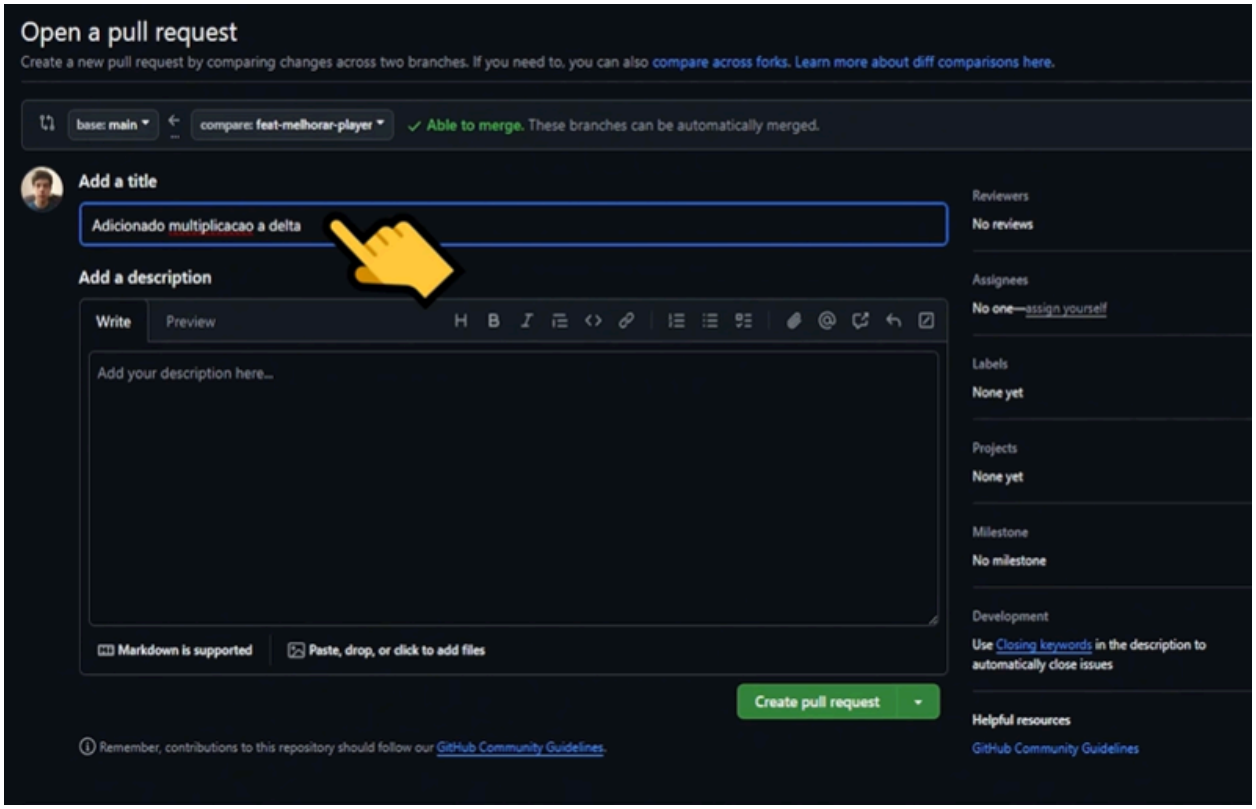
Todo projeto Git começa com a branch **main** (principal) que normalmente contém versões estáveis do produto. Como estamos produzindo recursos para o jogo, é recomendável criar uma **branch separada** para unir futuramente à main.



Na interface do Github Desktop, é possível ver que estamos na branch main, mas podemos criar uma nova branch. Assim, qualquer alteração no código será registrada nessa nova branch que pode ser visualizada no Github.

Pull Request

Para integrar essas novas mudanças da nova branch na branch principal, basta criar um **Pull Request:**



Para isso:

1. Criar um Pull Request
2. Revisar alterações
3. Realizar o merge

Merge Conflict

O Github irá verificar se o **merge** (mesclar) é possível. Se não for possível, pode ocorrer o erro **Merge Conflict** (Conflito de Mesclagem), tendo alterações no mesmo trecho e na mesma seção do código em diferentes commits, onde a ferramenta não sabe qual modificação será mantida.

O Github permite resolver esses conflitos no botão “**Resolve conflicts**”, mas o recomendado é solucionar na máquina local através de commits na máquina local.

Squash

É possível realizar o **Squash**, que junta todos os commits feitos em um único commit, mantendo o histórico mais organizado.

Git Large File Storage

Conforme o crescimento do projeto, você irá provavelmente lidar com **arquivos grandes** como **texturas**, **objetos 3D** ou **arquivos de áudios**. Isso pode se tornar um problema, pois todo repositório do Github possui um limite.

Para solucionar essa questão, surgiu o **Git Large File Storage** (https://git-lfs.com/), uma extensão do Git para lidar com arquivos maiores.

Normalmente esses arquivos grandes podem tornar o repositório lento, mas com o Git LFS podemos gerenciá-los de maneira eficiente, criando referências para eles, enquanto os arquivos reais são armazenados separadamente.

Infelizmente, não é possível usá-lo no Github Desktop, é necessário usar a versão de terminal. Clicando em: **Repository** → **Open in Command Prompt**. Se você não tiver o Git instalado, ele irá pedir a instalação.

Ao abrir o CMD, utilize os comandos:

- **Instalar o Git LFS:** git lfs install
- **Rastrear determinado arquivo grande:** git ifs track “[NOME]”
    - **Imagens:** git lfs track “*.png”
    - **Áudios:** git lfs track “*.wav”

Essas configurações ficaram salvas no arquivo **.gitattributes**.

Comandos extras:

- `git stash`, o que faz um salvamento local temporário do seu código
- `git reset`, que permite que você arrume seu código antes de fazer um commit
- `git bisect`, uma função que permite que você caçar maus commits
- `git squash`, o que permite que você combine seus commits
- `git rebase`, que permite aplicar alterações de uma ramificação para outra

RESUMO

O que é Versionamento de Código?

O versionamento de código é essencial para que desenvolvedores e equipes trabalhem de forma eficiente e organizada.

Ele permite:

- Rastrear todas as alterações feitas no código
- Recuperar versões anteriores em caso de erro
- Trabalhar em equipe simultaneamente no mesmo projeto

Por que usar Git?

O **Git** é um sistema de controle de versão distribuído que permite acompanhar todo o histórico de alterações de um projeto.

Principais vantagens:

- Controle completo do histórico
- Possibilidade de desfazer alterações
- Trabalho simultâneo em equipe
- Versionamento local (sem depender da internet)

Por que usar GitHub?

O **GitHub** é uma plataforma online de hospedagem de repositórios Git.

Ele funciona como:

- Armazenamento em nuvem para projetos
- Ferramenta de colaboração em equipe
- Histórico visual de commits e alterações

Também é amplamente conhecido como uma “rede social de desenvolvedores”.

Versionamento no Godot

Criando um novo projeto

Ao criar um projeto na Godot, selecione:

Controle de Versão → Git

Projeto existente

Para adicionar Git a um projeto já criado:

Projeto → Controle de Versão → Criar/Sobrescrever Metadados de Controle de Versão

---

Criando repositório local (GitHub Desktop)

1. Abra o GitHub Desktop
2. Clique em: **File → Add Local Repository**
3. Selecione a pasta do projeto
4. Clique em **Create a Repository**

Ao criar:

- Defina nome e descrição
- Adicione um `.gitignore` (selecionar Godot)
- Escolha uma licença

Licença

A licença define como seu código pode ser utilizado por outras pessoas.

Recomendação: **MIT License**, por ser simples e permissiva.

Publicando no GitHub

Após criar o repositório local:

1. Vá até a aba **Changes**
2. Registre as alterações com um **commit**
3. Clique em **Publish Repository**

Configurações disponíveis:

- Nome do repositório
- Descrição
- Público ou privado
- Organização (opcional)

Fluxo de Versionamento

Fluxo básico de trabalho:

1. Trabalhar no repositório local
2. Realizar alterações no projeto
3. Criar commits organizando as mudanças
4. Enviar para o GitHub

Estrutura do Repositório

- **README.md**: Documento principal com descrição do projeto
- **History**: Histórico de commits
- **Changes**: Alterações atuais

Atualizações do repositório

Antes de trabalhar:

- Clique em **Fetch origin**
- Se houver mudanças, utilize **Pull origin**

.gitignore

O arquivo `.gitignore` define quais arquivos não devem ser enviados ao repositório.

Exemplo importante no Godot:

- Pasta `.godot` (arquivos temporários)

Trabalho em Equipe

Branches

Branches são ramificações do projeto.

- **main**: versão estável
- **outras branches**: desenvolvimento de novas funcionalidades

Cada membro da equipe pode trabalhar em sua própria branch.

Pull Request

Para integrar mudanças à branch principal:

1. Criar um Pull Request
2. Revisar alterações
3. Realizar o merge

Merge Conflict

Ocorre quando há alterações no mesmo trecho de código.

Recomendação:

- Resolver localmente
- Criar novo commit com a correção

Squash

Permite juntar vários commits em um único commit, mantendo o histórico mais organizado.

Git Large File Storage (Git LFS)

Projetos podem conter arquivos grandes como:

- Texturas
- Áudios
- Modelos 3D

O Git LFS permite gerenciar esses arquivos sem sobrecarregar o repositório.

Link

🔗 https://git-lfs.com/

Como usar:

1. Instalar:

```
git lfs install
```

2. Rastrear arquivos:

```
git lfs track "*.png"
git lfs track "*.wav"
```

Essas configurações são salvas no arquivo `.gitattributes`.

Observação: o Git LFS não é totalmente suportado pelo GitHub Desktop, sendo necessário usar o terminal.

CONCLUSÃO

O uso de Git e GitHub é fundamental para garantir:

- Organização do projeto
- Segurança do código
- Colaboração eficiente em equipe

A adoção dessas ferramentas permite que o desenvolvimento seja mais profissional, escalável e confiável.

VEJA TAMBÉM:

Repositório contendo estudos de Godot através do curso no Youtube do professor Rafael Forbeck: https://github.com/ReverseYuji049/projeto-1.git